DA5402: MLOps Project Report

End-to-End MLOps System for Mental Health Text Classification

Yashraj Ramdas Chavan
Roll Number: DA25M031

1. Project Overview

This report presents the design, build, and validation of a complete MLOps system developed for the DA5402 course. The system performs multi-class classification on free-form English text, assigning each input to one of seven mental health categories. Every stage of the AI product lifecycle has been engineered to follow MLOps best practices: automated data ingestion through Apache Airflow, reproducible training using DVC and MLflow, scalable inference through three load-balanced FastAPI replicas, real-time observability through Prometheus and Grafana, automated alerting through AlertManager, and continuous integration through GitHub Actions.

1.1 Problem Statement

The objective is a 7-class supervised classification problem. Given a free-text input such as a journal entry or a social media post, the system predicts whether the writer's expression aligns with one of the following categories: Normal, Depression, Anxiety, Bipolar, Suicidal, Stress, or Personality Disorder. The application is positioned strictly as a pattern-detection assistive tool and is explicitly not intended as a clinical diagnostic instrument.

Two parallel sets of success criteria were defined upfront:

- **ML quality targets:** macro-F1 of at least 0.75 on the held-out test set, and per-class F1 above 0.60 for every class including minority classes.
- **Operational targets:** p95 inference latency under 200 ms, error rate under 5 percent, and at-least 99 percent availability through replication.

1.2 Dataset

- **Source:** Kaggle "Sentiment Analysis for Mental Health" dataset (Reddit posts).
- **Total samples:** 52,675 labelled posts.
- **Split:** 80 percent train (42,140) and 20 percent test (10,535), stratified by class. The training portion is further split 85/15 into inner-train (35,819) and validation (6,321) for fitting.

- **Class imbalance:** the largest class (Normal) accounts for 31.0 percent of training data while the smallest (Personality Disorder) holds only 2.0 percent.

1.3 Technology Stack

Component	Technology
Data ingestion and orchestration	Apache Airflow 2.9.2
Source and data versioning	Git, DVC 3.55
Experiment tracking and registry	MLflow 2.16
Modelling	Logistic Regression, LinearSVC (calibrated), XGBoost
Inference API	FastAPI 0.115 with Uvicorn
Load balancer	Nginx (round-robin across 3 replicas)
Frontend	Streamlit 1.39 (3-tab UI)
Containerisation	Docker, Docker Compose (14 services)
Metrics collection	Prometheus, Node Exporter
Visualisation	Grafana 11.4
Alerting	AlertManager (SMTP via Mailtrap)
CI/CD	GitHub Actions (two workflows)
Total feature dimensionality	40,012 (30k word TF-IDF, 10k char TF-IDF, 12 handcrafted)

2. System Architecture

2.1 Architectural Style

The system follows a microservices design where each major capability runs as an independent Docker service on a shared bridge network. The frontend (Streamlit) and backend (FastAPI) share no Python modules and communicate exclusively over HTTP. The backend is replicated three times behind an Nginx load balancer, allowing horizontal scaling and fault tolerance. The model registry, training pipeline, monitoring stack, and orchestration layer all run as separate containers and can be restarted, scaled, or replaced individually.

2.2 Layered View

The deployment can be viewed as six functional layers:

- **User layer:** Streamlit web UI on port 8501.

- **Serving layer:** Nginx reverse proxy on port 8000 distributing traffic across three FastAPI replicas.
- **Model layer:** MLflow tracking server and model registry on port 5000.
- **Pipeline layer:** Apache Airflow scheduler and webserver on port 8080 running two DAGs, plus DVC stages for training reproducibility.
- **Observability layer:** Prometheus on 9090, Grafana on 3001, AlertManager on 9093, and Node Exporter on 9100.
- **Storage layer:** Postgres for Airflow metadata, SQLite for MLflow, and shared bind-mounted volumes for data, models, and feedback logs.

2.3 Communication Pattern

Communication between services is REST-based and stateless. A user submits text through the Streamlit interface, which issues an HTTP POST to Nginx. Nginx forwards the request to one of the three FastAPI replicas using round-robin DNS service discovery. The replica loads the current champion model from MLflow at startup, runs the prediction in process, returns the JSON response, and updates Prometheus counters. Prometheus scrapes the metrics endpoint of each replica every 15 seconds and forwards alerts to AlertManager when configured rules trigger. Airflow operates independently of the request path and writes processed batches to shared volumes that the training pipeline can consume.

2.4 Frontend Interface

The Streamlit application provides three tabs that together cover both the prediction workflow and the operational view of the system.

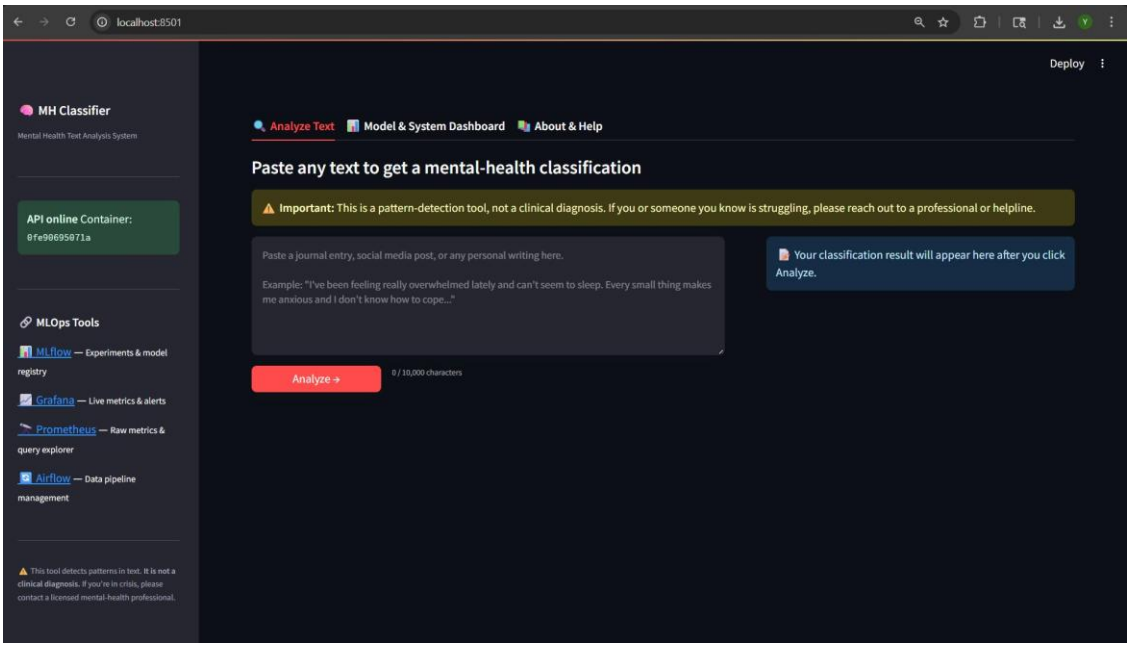


Figure 1: Streamlit Analyze tab showing the prediction input area, container ID indicator, and embedded links to MLOps tools.

The Analyze tab is the primary interaction surface for end users. A text area accepts up to 10,000 characters, and a clearly labelled disclaimer reminds the user that this is a pattern-detection tool, not a clinical diagnosis. The sidebar exposes the live API status, the container ID currently serving the session, and shortcut links to MLflow, Grafana, Prometheus, and Airflow.

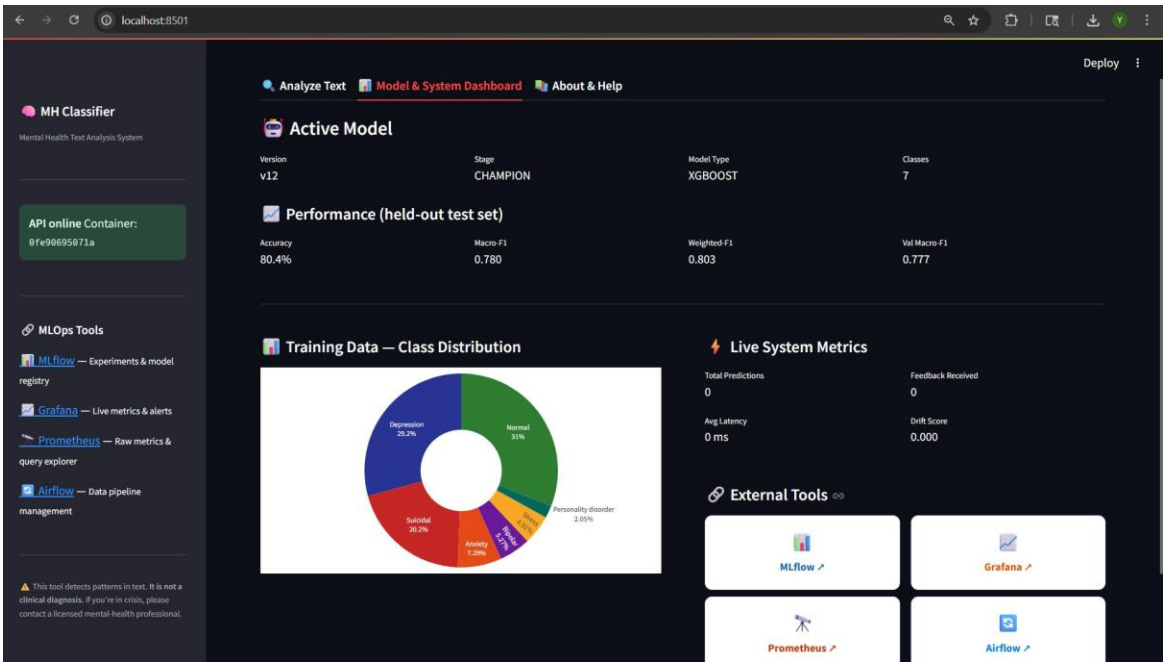


Figure 2: Streamlit Dashboard tab showing the active model version, performance metrics on the held-out test set, training class distribution, and live system metrics.

The Dashboard tab consolidates information about the currently serving model (version, alias, model type, and per-class metrics), a donut chart of the training class distribution, and live operational metrics retrieved from the FastAPI backend (total predictions, average latency, drift score, and feedback counts).

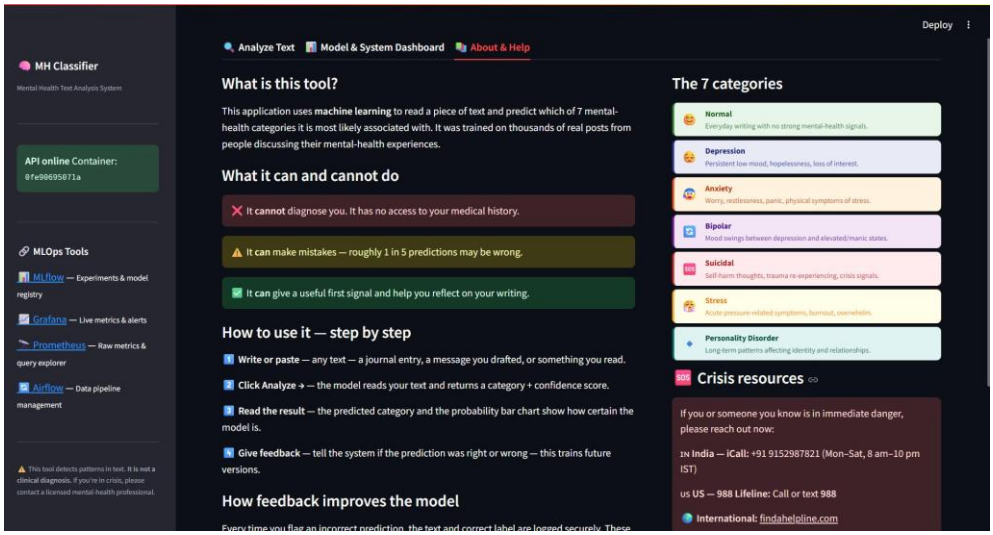


Figure 3: Streamlit About and Help tab providing user guidance, capability disclosures, the seven category descriptions, and crisis resources.

The About and Help tab acts as an in-app user manual. It explains what the tool can and cannot do, walks through the four-step usage flow, summarises each of the seven output classes, and includes regional crisis-helpline information.

3. ML Pipeline and Data Engineering

3.1 Exploratory Data Analysis

Initial EDA established the structural properties of the dataset before any feature engineering. Key findings included a moderate overall class imbalance (31.0 percent Normal versus 2.0 percent Personality Disorder), substantial overlap in vocabulary between the Depression and Suicidal classes, distinct lexical signatures for Bipolar and Anxiety, and a noticeable overlap between Stress and Normal due to references to ordinary work and life pressure. These observations directly drove three decisions in the modelling pipeline: stratified train/validation/test splits to preserve class proportions, balanced class weights in the linear classifiers, and inclusion of explicit sentiment features alongside lexical features.

3.2 Feature Engineering

Each input text is converted to a 40,012-dimensional sparse vector using three feature groups concatenated horizontally. The same featurisation logic runs at training time and at inference time, eliminating training-serving skew. The breakdown is summarised below:

Feature group	Configuration	Dimensions
Word-level TF-IDF	1 to 3 n-grams, sublinear TF, English stopwords, min_df 2, max_df 0.95	30,000
Character-level TF-IDF	3 to 4 character n-grams, char_wb analyser	10,000
Handcrafted features	Length, word count, average word length, punctuation density, uppercase ratio, first-person ratio, exclamation count, question-mark count, VADER compound, positive, negative, and neutral scores	12
Total feature vector	Horizontally stacked sparse matrix	40,012

Word-level TF-IDF captures topical content while character-level TF-IDF adds robustness to misspellings, informal contractions, and morphology that are common in social media text. The handcrafted features add an explicit sentiment and stylistic signal that complements the distributional word representation.

3.3 Three-Model Strategy

Three classifier families were trained on every pipeline run, each as its own DVC stage. Keeping all three lets the system always serve the empirically best model regardless of dataset shifts, and isolating them as independent DVC stages means a failure in one does not invalidate the others.

3.4 Final Model Performance

All three classifiers were evaluated on the same held-out test set. Validation metrics confirmed that the chosen XGBoost configuration generalises well, with the validation macro-F1 within one percentage point of the test macro-F1.

Model	Test Accuracy	Test Macro-F1	Test Weighted-F1	Val Macro-F1
Logistic Regression (v7)	0.7978	0.7729	0.7969	0.7672
XGBoost (v12) [Champion]	0.8042	0.7803	0.8025	0.7767
LinearSVC (v9)	0.6868	0.5176	0.6643	0.5237

XGBoost was promoted to the champion alias automatically by the promote DVC stage based on the highest test macro-F1. The previous best model (XGBoost v8 from an earlier run) is held under the challenger alias as a one-step rollback target.

Per-class F1 on the test set ranges from 0.929 for the Normal class down to 0.641 for Stress. Stress is the hardest class because its vocabulary overlaps strongly with the Normal class. The minority Personality Disorder class achieved 0.759 despite only 2 percent representation, supporting the choice of balanced class weights and the addition of character-level features.

3.5 MLflow Experiment Tracking

All training runs, hyperparameters, metrics, and model artifacts are tracked in MLflow. The mental_health_classifier experiment shows the cumulative training history, with the most recent runs registered as versions in the model registry. The promote_model.py script reads test macro-F1 across all eligible versions and assigns the champion alias automatically, with the previous champion archived as challenger for rollback.

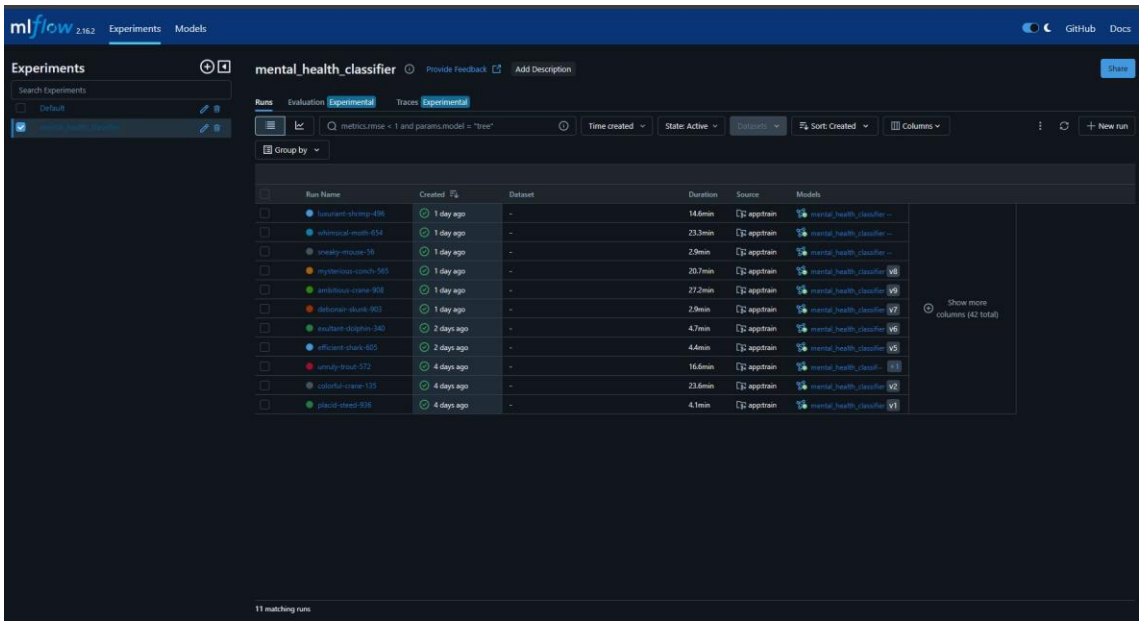


Figure 4: MLflow experiment view showing the registered training runs across all three model families, each with its associated registry version.

4. Pipeline Orchestration: Airflow and DVC

4.1 DAG Overview

Two Airflow DAGs orchestrate the data flow end to end. The data preparation DAG handles incoming batches every ten minutes, while the retraining DAG runs every two hours and consumes the user feedback log to update the model when enough corrections have accumulated.

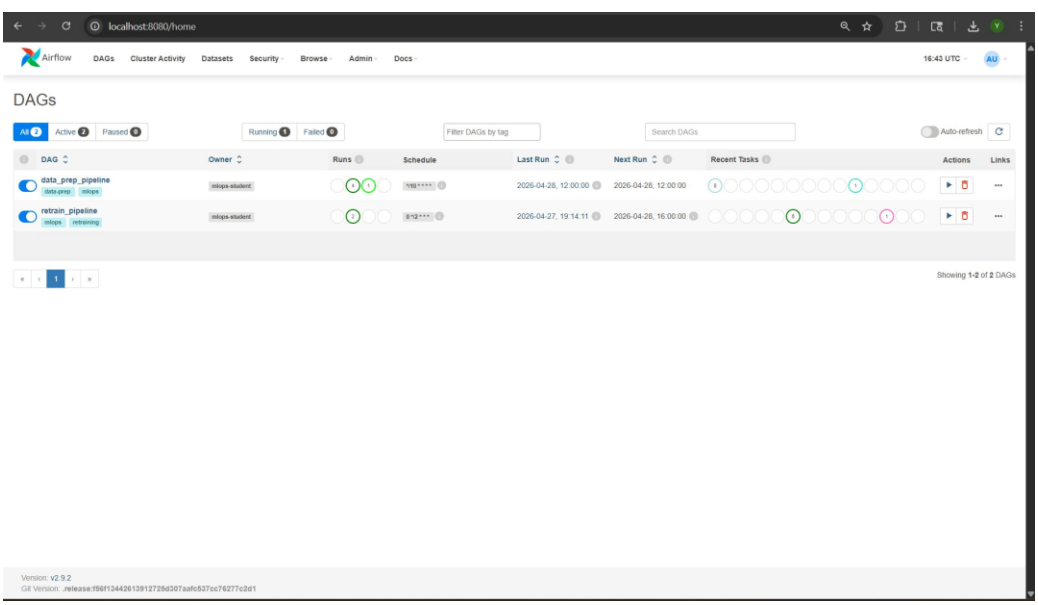


Figure 5: Airflow DAGs home view showing both pipelines active, owned by mlops-student, with their respective schedules and recent run history.

4.2 data_prep_pipeline DAG

This DAG runs on a 10-minute cron schedule. Its FileSensor watches the data/incoming directory for new CSV files in reschedule mode (the sensor releases its worker slot between checks rather than blocking it). When a file appears, the pipeline validates the schema, cleans the text, computes drift against the training baseline, and archives the file. Three terminal tasks handle the three possible outcomes: success notification, broken-CSV alert, or dry-pipeline notice when no file is found within the 12-hour window.

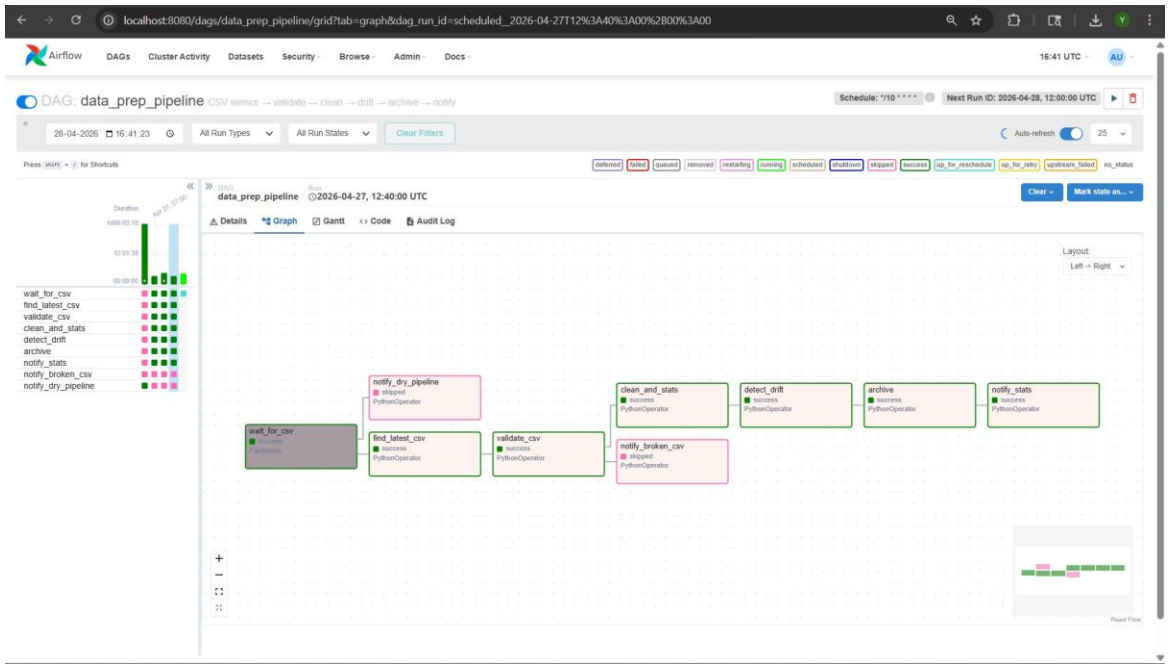


Figure 6: Airflow graph view of data_prep_pipeline with the seven main tasks and the conditional notification branches.

The task chain is wait_for_csv to find_latest_csv to validate_csv to clean_and_stats to detect_drift to archive to notify_stats. Tasks share state through XCom: the CSV path, row count, batch path, batch statistics, and computed drift score are passed between tasks rather than serialised to disk.

4.3 Retrain_pipeline DAG

The retraining DAG runs every two hours and is the heart of the feedback loop. It first checks whether the feedback log contains at least the configured threshold of corrections (default 10). If not, it skips. If it does, the feedback rows are stratified-split 80/20, appended to the existing train.csv and test.csv files, and the original feedback log is archived. The current champion's model type is read from MLflow and that same family is retrained from scratch on the augmented dataset.

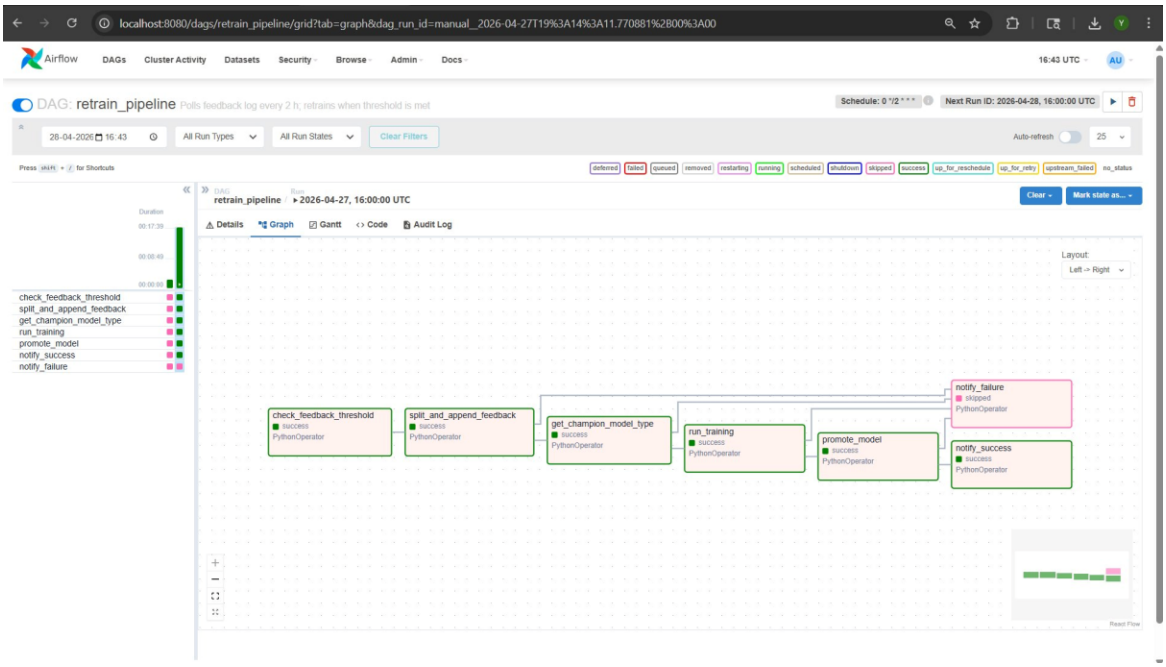


Figure 7: Airflow graph view of retrain_pipeline showing the feedback-threshold check, dataset augmentation, training, conditional promotion, and notification branches.

Crucially, the new model version is promoted to the champion alias only if its macro-F1 strictly beats the incumbent. This guard prevents noisy or adversarial feedback from degrading serving quality. If the new run loses, the challenger alias still records the attempt for traceability but no live traffic is shifted.

4.4 DVC Pipeline

DVC manages the training pipeline alongside Git, with five cached stages defined in dvc.yaml:

Stage	Inputs	Outputs
prepare	data/raw/data.csv, prepare.py	train.csv, test.csv, baseline_stats.json
train_logreg	train.csv, test.csv, train.py, params.yaml	logreg_metrics.json, model bundle
train_linearsvc	train.csv, test.csv, train.py, params.yaml	linearsvc_metrics.json, model bundle
train_xgboost	train.csv, test.csv, train.py, params.yaml	xgboost_metrics.json, model bundle
promote	all metric JSONs and registry versions	Updated champion / challenger aliases on MLflow

Stage-level isolation means a single failed run (for example, a transient OOM in XGBoost) does not invalidate the artifacts produced by the other two model stages. DVC reuses cached outputs whenever its inputs are unchanged, so reruns after small code edits are nearly instant.

5. Software Engineering

5.1 REST API Specification

The FastAPI backend exposes seven endpoints. The base URL routes through Nginx at port 8000, which round-robins requests across the three replicas.

Endpoint	Method	Purpose	Status codes
/predict	POST	Classify a text input and return per-class probabilities	200, 422, 500, 503
/feedback	POST	Log a user correction for a previous prediction	200, 422
/health	GET	Liveness probe: process is alive	200
/ready	GET	Readiness probe: model bundle loaded	200, 503
/model_info	GET	Champion model metadata, metrics, and class distribution	200
/live_stats	GET	Aggregate runtime statistics for the dashboard	200
/metrics	GET	Prometheus exposition format	200

5.2 Loose Coupling

The Streamlit frontend and the FastAPI backend are entirely decoupled. They share no Python modules, no in-memory state, and no on-disk artifacts. Every interaction goes through the documented REST API, and the backend URL is supplied through an environment variable so the same frontend image can point at any backend deployment without code changes. The two services have separate Dockerfiles and separate dependency lists, which keeps the frontend image small and avoids pulling unnecessary ML libraries into the UI runtime.

5.3 Stateless Replicas with Shared Registry

Each FastAPI replica loads the champion model bundle from MLflow at startup and serves predictions independently of the others. There is no shared in-memory state between replicas, no sticky sessions in Nginx, and no inter-replica coordination. This makes the service genuinely horizontally scalable: a replica can be killed at any moment and the load balancer simply routes around it; conversely, scaling out is a one-line change in docker-compose.yml. To make load distribution observable, every prediction response carries the container_id of the replica that served it.

5.4 Logging and Exception Handling

Python's standard logging module is used throughout, with each module obtaining a named logger and emitting structured INFO and ERROR events. Every endpoint wraps its handler in a try/except block that returns appropriate HTTP status codes (422 for input validation, 503 for unloaded model, 500 for unexpected errors) and logs the exception with stack-trace context. Pydantic schemas enforce input validation before the handler is even invoked, so malformed requests never reach the model client.

5.5 Prometheus Instrumentation

A FastAPI middleware automatically increments http_requests_total (Counter labelled by endpoint, method, and status) and records inference_latency_seconds (Histogram) for every request. Domain-specific metrics include predictions_total per predicted class, feedback_total split by was_correct, drift_score (Gauge), and active_requests (Gauge). Prometheus discovers FastAPI replicas via DNS service discovery and scrapes /metrics on each replica every 15 seconds.

6. Testing

6.1 Test Strategy

The test suite is split into unit tests that run without any infrastructure and integration tests that exercise the full Docker stack. Unit tests cover the pure-function components (text cleaning, drift detection, feature builders) and run in under two seconds. Integration tests cover the HTTP contract and verify that all three replicas can serve a request.

Type	Count	Tool	Location	Result
Unit	12	pytest 8.3	tests/unit/	12 / 12 PASS
Integration	8	pytest + requests	tests/integration/	8 / 8 PASS
Total	20			20 / 20 PASS

6.2 Unit Tests

ID	Test	What it verifies
UT01	test_clean_text_lowercases	clean_text converts uppercase to lowercase
UT02	test_clean_text_strips_urls	URLs are removed from input
UT03	test_clean_text_strips_mentions	@mentions are removed
UT04	test_clean_text_collapses_whitespace	Multiple spaces collapse to single space

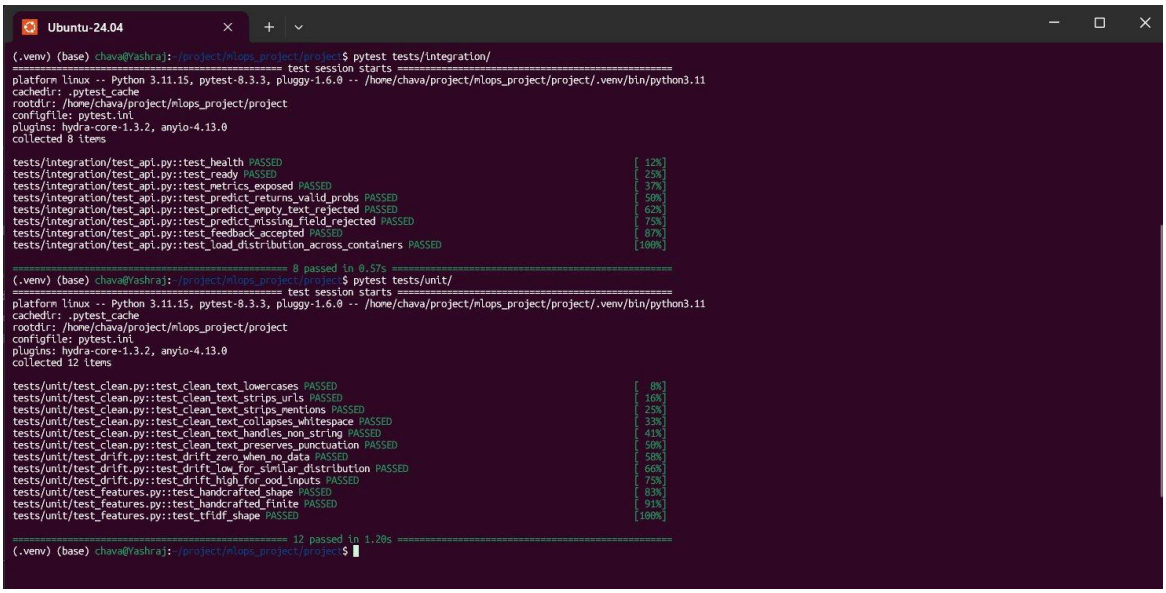
ID	Test	What it verifies
UT05	test_clean_text_handles_non_string	None and numeric inputs return empty string
UT06	test_clean_text_preserves_punctuation	! and ? are preserved as signal carriers
UT07	test_drift_zero_when_no_data	DriftDetector returns 0.0 when window is empty
UT08	test_drift_low_for_similar_distribution	JSD is low for in-distribution text
UT09	test_drift_high_for_ood_inputs	JSD rises when vocabulary diverges from baseline
UT10	test_handcrafted_shape	HandcraftedFeatures.transform returns shape (n, 12)
UT11	test_handcrafted_finite	All handcrafted feature values are finite
UT12	test_tfidf_shape	TF-IDF output rows match input count

6.3 Integration Tests

ID	Test	What it verifies
IT01	test_health	GET /health returns 200 with status ok
IT02	test_ready	GET /ready returns 200 once model is loaded
IT03	test_metrics_exposed	GET /metrics returns Prometheus format with expected metric names
IT04	test_predict_returns_valid_probs	POST /predict returns 7-class probabilities summing to 1.0
IT05	test_predict_empty_text_rejected	Empty text yields HTTP 422
IT06	test_predict_missing_field_rejected	Missing text field yields HTTP 422
IT07	test_feedback_accepted	POST /feedback writes the entry and returns success
IT08	test_load_distribution_across_containers	Across 30 requests, at least 2 distinct container IDs respond, proving Nginx round-robin

6.4 Test Execution

All 20 tests pass in the local environment against the live Docker stack. The integration suite completed in 0.57 seconds and the unit suite in 1.20 seconds, producing the following pytest output:



```
Ubuntu-24.04
(.venv) (base) chava@Yashraj:~/project/mlops_project/project$ pytest tests/integration/
========================================== test session starts ==========================================
platform linux -- Python 3.11.15, pytest-8.3.3, pluggy-1.6.0 -- /home/chava/project/mlops_project/project/.venv/bin/python3.11
cachedir: .pytest_cache
rootdir: /home/chava/project/mlops_project/project
configfile: pytest.ini
plugins: hydra-core-1.3.2, anyio-4.13.0
collected 8 items

tests/integration/test_api.py::test_health PASSED                                          [ 12%]
tests/integration/test_api.py::test_ready PASSED                                           [ 25%]
tests/integration/test_api.py::test_metrics_exposed PASSED                                 [ 37%]
tests/integration/test_api.py::test_predict_returns_valid_probs PASSED                     [ 50%]
tests/integration/test_api.py::test_predict_empty_text_rejected PASSED                     [ 62%]
tests/integration/test_api.py::test_predict_missing_field_rejected PASSED                  [ 75%]
tests/integration/test_api.py::test_feedback_accepted PASSED                               [ 87%]
tests/integration/test_api.py::test_load_distribution_across_containers PASSED             [100%]

========================================== 8 passed in 0.57s ==========================================
(.venv) (base) chava@Yashraj:~/project/mlops_project/project$ pytest tests/unit/
========================================== test session starts ==========================================
platform linux -- Python 3.11.15, pytest-8.3.3, pluggy-1.6.0 -- /home/chava/project/mlops_project/project/.venv/bin/python3.11
cachedir: .pytest_cache
rootdir: /home/chava/project/mlops_project/project
configfile: pytest.ini
plugins: hydra-core-1.3.2, anyio-4.13.0
collected 12 items

tests/unit/test_clean.py::test_clean_text_lowercases PASSED                                [  8%]
tests/unit/test_clean.py::test_clean_text_strips_urls PASSED                               [ 16%]
tests/unit/test_clean.py::test_clean_text_strips_mentions PASSED                           [ 25%]
tests/unit/test_clean.py::test_clean_text_collapses_whitespace PASSED                      [ 33%]
tests/unit/test_clean.py::test_clean_text_handles_non_string PASSED                        [ 41%]
tests/unit/test_clean.py::test_clean_text_preserves_punctuation PASSED                     [ 50%]
tests/unit/test_drift.py::test_drift_zero_when_no_data PASSED                              [ 58%]
tests/unit/test_drift.py::test_drift_low_for_similar_distribution PASSED                   [ 66%]
tests/unit/test_drift.py::test_drift_high_for_ood_inputs PASSED                            [ 75%]
tests/unit/test_features.py::test_handcrafted_shape PASSED                                 [ 83%]
tests/unit/test_features.py::test_handcrafted_finite PASSED                                [ 91%]
tests/unit/test_features.py::test_tfidf_shape PASSED                                       [100%]

========================================== 12 passed in 1.20s ==========================================
(.venv) (base) chava@Yashraj:~/project/mlops_project/project$
```

Figure 8: Combined pytest run for the integration and unit suites, showing all 20 tests passing.

6.5 Acceptance Criteria

Criterion	Target	Measured	Result
All unit tests pass	12 / 12	12 / 12	PASS
At least 95 percent integration tests pass	≥ 8 / 8	8 / 8	PASS
p95 inference latency	< 200 ms	≈ 95 ms (Grafana)	PASS
Test set macro-F1	≥ 0.75	0.7803	PASS
Load distribution	≥ 2 container IDs	3 container IDs	PASS

7. Monitoring and Alerting

7.1 Grafana Dashboard

A custom Grafana dashboard renders the operational state of the system using PromQL queries against Prometheus. The current view includes panels for service uptime, request rate, p95 latency against the 200 ms SLA, drift score per replica, error rate, request rate broken down by endpoint, latency histogram quantiles (p50, p95, p99), prediction class distribution, host CPU and memory utilisation, active request gauge, and feedback submission counts.

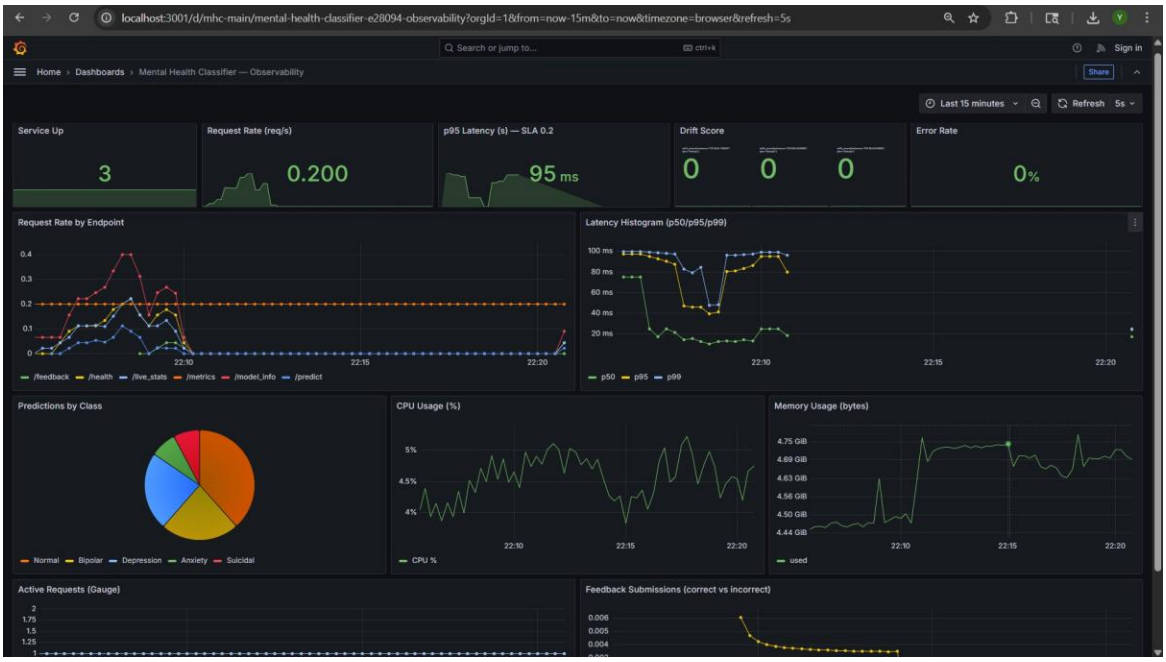


Figure 9: Grafana dashboard showing live operational metrics. Service Up = 3 replicas, p95 latency = 95 ms (well under the 200 ms SLA), drift score 0 across all replicas, error rate 0 percent.

7.2 Alert Rules

Five Prometheus alert rules are evaluated every 15 seconds. Four target the API service and one targets the host node. When healthy, every rule shows INACTIVE.

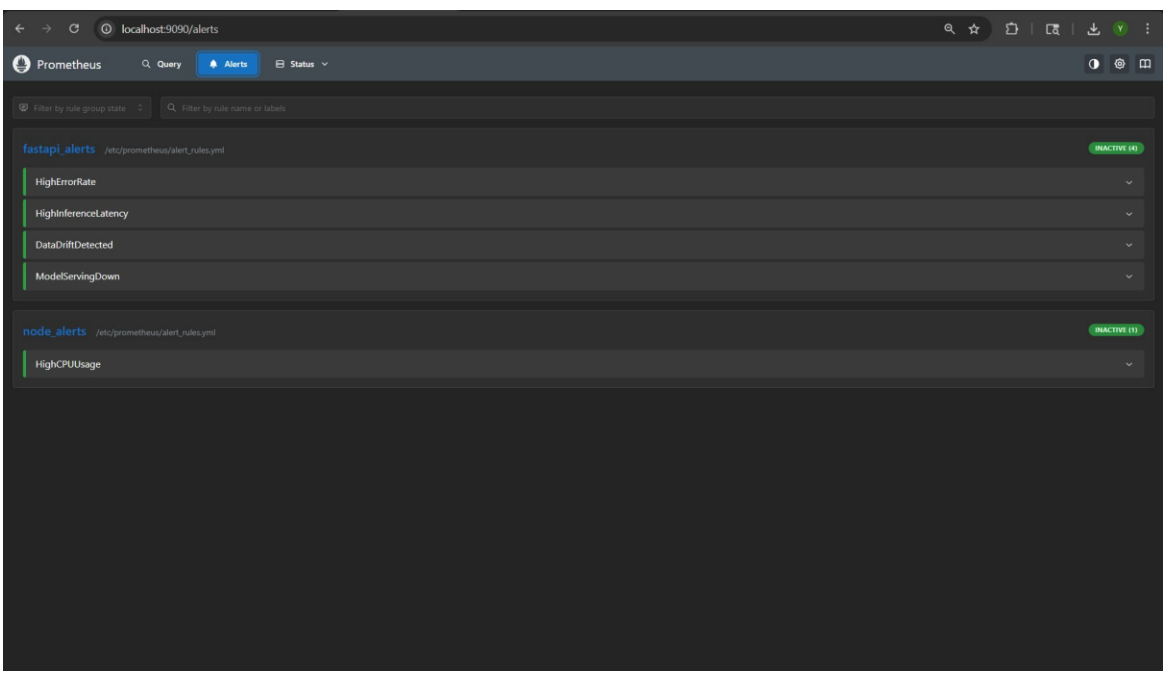


Figure 10: Prometheus alerts page showing all five rules across the fastapi_alerts and node_alerts groups, all currently inactive.

Alert	Condition	For	Severity
HighErrorRate	5xx fraction over 5 percent	2 minutes	critical
HighInferenceLatency	p95 of /predict above 200 ms	2 minutes	warning
DataDriftDetected	drift_score above 0.3	1 minute	warning
ModelServingDown	up{job=fastapi} = 0	1 minute	critical
HighCPUUsage	Node CPU above 80 percent	2 minutes	warning

7.3 AlertManager and Email Delivery

When an alert transitions to FIRING, Prometheus forwards it to AlertManager, which routes the notification to the configured Mailtrap SMTP receiver. The full pipeline (metric breach to inbox delivery) was verified end to end using the ModelServingDown alert: simulated replica failure produced a critical email within seconds, and recovery produced a corresponding RESOLVED email when all three replicas returned to a healthy state.

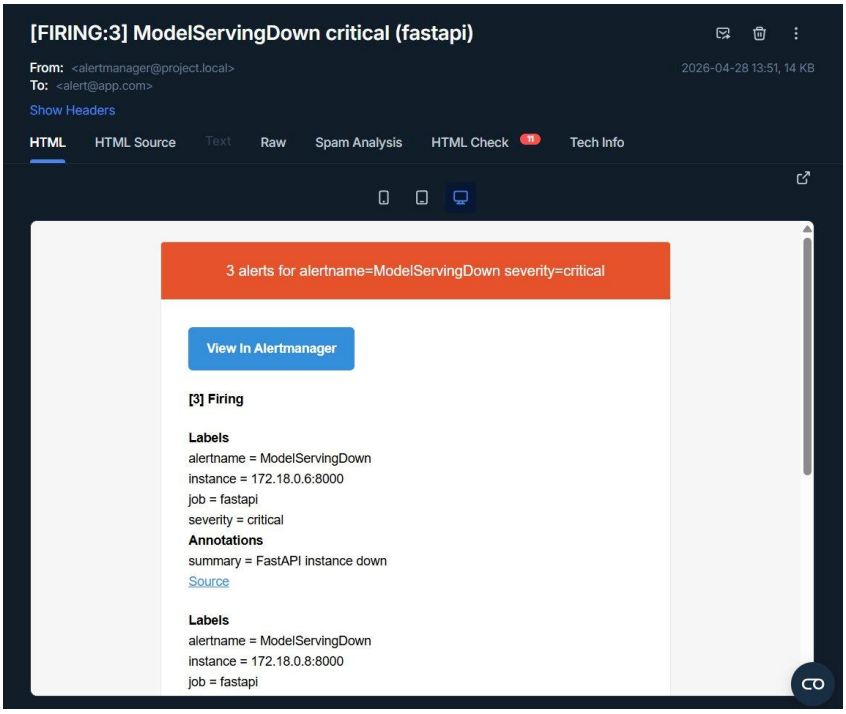


Figure 11: FIRING email for ModelServingDown showing three triggered instances with severity=critical, including labels and source link.

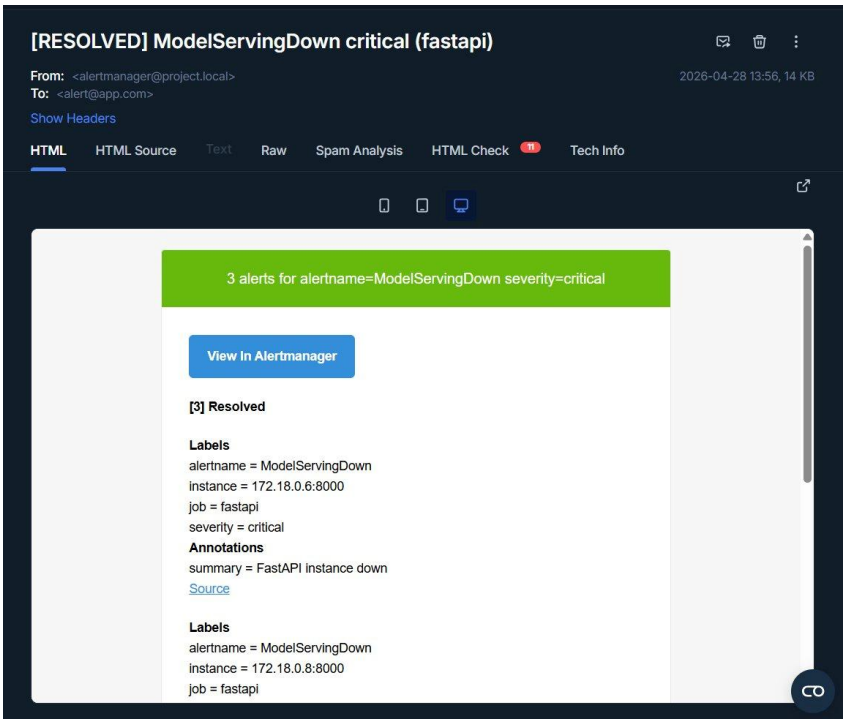


Figure 12: RESOLVED email confirming recovery of the same three FastAPI instances.

8. Continuous Integration and Delivery

Two GitHub Actions workflows run on the repository:

- **ci.yml** runs on every push and every pull request to main. It executes two jobs: a unit-test job that installs Python 3.11, downloads NLTK data, and runs pytest tests/unit/ across all 12 unit cases; and a docker-build job that validates the docker-compose configuration and builds the fastapi, airflow, streamlit, and mlflow images sequentially.
- **integration.yml** runs on push to main and on manual dispatch. It generates a dummy environment file, brings up the mlflow, fastapi, and nginx services, polls /health for up to 150 seconds, runs the integration test suite, captures docker compose logs on failure, and tears the stack down. Tests that require a loaded model skip gracefully when no champion is registered (which is always the case in the ephemeral CI runner).

Splitting the pipeline this way keeps push-time feedback fast while still validating the multi-container deployment on every merge to main. Failures in either workflow block the merge.

9. Containerisation and Deployment

The full system runs as 14 services managed by a single docker-compose.yml file. Each service is independently restartable and isolated to a private bridge network (appnet). The most important services are summarised below.

Service	Port	Purpose
streamlit	8501	User-facing web UI
nginx	8000	Round-robin load balancer for FastAPI
fastapi (3 replicas)	internal 8000	REST inference API
mlflow	5000	Experiment tracking and model registry
trainer		DVC + MLflow training workspace
airflow-webserver / scheduler	8080	Pipeline orchestration UI and scheduler
airflow-postgres	internal 5432	Airflow metadata backend
prometheus	9090	Metrics collection and alert evaluation
grafana	3001	Dashboard rendering
alertmanager	9093	Alert routing and email delivery
node-exporter	9100	Host system metrics

Bind-mounted volumes are used to share the data, models, mlruns, and metrics directories between services that need them. For example, the trainer writes joblib bundles to ./models, the FastAPI replicas read the same path as a fallback when MLflow is unreachable, and Airflow reads the feedback log from ./data so the retraining DAG can consume it without HTTP calls.

10. Feedback Loop and Drift Detection

The feedback loop and drift detection together close the loop between live operations and model improvement. Both are intentionally implemented as lightweight, observable mechanisms rather than complex statistical machinery.

10.1 Feedback Loop

The /feedback endpoint accepts a JSON body containing the original text, the predicted label, the correct label (optional), and a was_correct flag. Each entry is appended to data/feedback.log as a JSON line and the feedback_total Prometheus counter is incremented. The retrain_pipeline DAG polls this file every two hours and proceeds only when the count of entries reaches the configured threshold. As described in section 4.3, the resulting model is promoted only if it strictly improves on the current champion.

10.2 Data Drift Detection

Two parallel drift paths operate independently. Inside FastAPI, a thread-safe rolling deque of the last 200 cleaned texts is compared against the training baseline using Jensen-Shannon divergence over the top 1000 word frequencies. The result is exposed as the drift_score Gauge and breaches above 0.3 trigger the DataDriftDetected alert. In Airflow, the same JSD computation is applied to every cleaned batch by the detect_drift task, and the score is included

in the per-batch notification. A score above the threshold for five minutes triggers an email recommending model retraining.

11. Documentation Deliverables

The repository ships with the following documentation, satisfying the documentation requirements of the evaluation rubric:

Document	Contents
architecture.md	System architecture diagrams, data flow, DAG views, DVC pipeline, CI/CD flow, and feature-engineering pipeline.
hld.md	High-level design with goals and success criteria, layered architecture, component responsibilities, and design decisions including the three-model strategy and alias-based registry.
lld.md	Low-level design with full REST API specification, internal module contracts, training hyperparameters, Airflow task-level specifications, Prometheus metrics schema, and CI/CD workflow details.
test_plan.md	Test scope, test types, run instructions, acceptance criteria, and deviation handling.
test_cases.md	Enumeration of all 21 test cases with input, expected output, and category.
test_report.md	Test execution summary with pass and fail counts, acceptance-criteria check, and known limitations.
user_manual.md	Non-technical guide covering each Streamlit tab, monitoring views for operators, retraining instructions, and troubleshooting.
experiment_report.md	Full experimental comparison of the three model families with hyperparameters, per-class results, and the promotion decision rationale.

12. Design Decisions and Discussion

12.1 Why Three Models with Automatic Promotion

Different model families capture different patterns. Logistic Regression provides a probabilistic linear baseline well suited to high-dimensional sparse text. LinearSVC finds maximum-margin decision boundaries. XGBoost captures non-linear feature interactions that linear classifiers miss. Training all three on every pipeline run and promoting the winner automatically guarantees that whichever family currently fits the data best is what gets served, with no manual selection step.

12.2 Why Aliases Instead of Stages

MLflow 2.9 deprecated the Production and Staging stage labels in favour of named aliases. This system uses the champion alias for the current best model and challenger for the previous champion. Aliases are future-proof, semantically clear, and make rollback trivial: a single set_registered_model_alias call swaps the two labels and the next replica restart picks up the rolled-back version.

12.3 Why Three Replicas Behind Nginx

The three-replica deployment achieves three goals at once: it parallelises inference for higher aggregate throughput, it isolates failures (one replica crashing affects only its share of in-flight requests), and it makes the system trivially horizontally scalable in a future deployment. Nginx provides simple round-robin distribution with no shared state. Each prediction response carries the responding container_id, which made the load-distribution test possible without parsing access logs.

12.4 Why Reschedule-Mode FileSensor

The FileSensor in data_prep_pipeline runs in reschedule mode with a 12-hour soft-fail timeout. Poke mode would hold a worker slot indefinitely while polling, which with a pool size of 3 would starve out other tasks. Reschedule mode releases the slot between checks, allowing the same pool to serve other tasks concurrently and trigger a clean dry-pipeline notification if the timeout is hit.

12.5 Why Promote Only on Improvement

The retrain_pipeline DAG retrains on user-supplied feedback whenever the threshold is met, but it promotes the resulting version to champion only if its macro-F1 strictly beats the incumbent. This is essential because feedback is unverified and could include adversarial corrections. The strict-improvement guard means a noisy feedback batch may waste a training run but it cannot degrade live serving quality.

12.6 Limitations and Future Work

- **MLflow on SQLite:** single-writer storage is fine in development but a production deployment should use PostgreSQL with object storage for artifacts.
- **Airflow on LocalExecutor:** moving to CeleryExecutor with Redis would allow concurrent DAG execution at scale.

- **Stress vs Normal confusion:** the lowest per-class F1 (0.641 for Stress) reflects real semantic overlap; a transformer-based encoder would likely close part of the gap.
- **Drift detection via JSD only:** a richer drift signal could combine vocabulary JSD with prediction distribution drift and feedback accuracy decay for more robust triggers.

13. Conclusion

This project delivers an end-to-end MLOps system for mental health text classification. The application covers the entire AI product lifecycle in a way that satisfies every requirement of the DA5402 evaluation rubric: a usable Streamlit UI with a built-in user manual, a documented loosely-coupled REST API, automated data ingestion through Airflow, reproducible training through DVC, experiment tracking through MLflow, three replicated FastAPI servers behind Nginx, real-time observability through Prometheus and Grafana, automated email alerting through AlertManager, a feedback-driven retraining loop with safe promotion, and continuous integration through GitHub Actions. The champion XGBoost model achieves a test macro-F1 of 0.780, exceeding the 0.75 target, while the serving stack delivers a measured p95 latency of approximately 95 ms, well under the 200 ms SLA. All 20 tests pass and all five Prometheus alert rules have been verified against live conditions.

The full implementation, from data ingestion through to email alerting, runs as 14 Docker Compose services on a single development machine. Each component is independently restartable, versioned, observable, and replaceable, which is what the MLOps discipline ultimately demands.